

```

1  /* =====
2      sonifikation.js – Kapitel als Tonstück, über Strudel
3
4      Liefert NUR den Ton zum Play-Button der Graph-Ansicht; die Spine läuft
5      parallel mit derselben Gesamtdauer. Zeitbasiert, nicht scroll-gekoppelt.
6
7      Zwei Modelle nebeneinander:
8      – Stationsmodell (Kapitel 02–18, und Kapitel 1 bei MODUS 'stationen'):
9        eine Tonstufe je Station, drei F-Wert-Kategorien als drei Layer in
10       c-moll. Zeitplan aus Gehstrecke und Annotationsdichte je Station
11       (kapitel01-sonifikation.json).
12      – Elementmodell (Prototyp, nur Kapitel 1): ein Klang je Element, drei
13       Gefühlskategorien als Instrumente, Tonhöhe aus dem Kreisradius. Siehe
14       den Block "Prototyp" weiter unten.
15  ===== */
16
17  // --- Modulkapselung -----
18  // 17 von 21 Namen intern, 4 exportiert. Konvention: docs/architektur.md.
19  (function () {
20
21      // Dieselben zwei CDN-Quellen, die strudel.cc selbst lädt. @strudel/web bringt
22      // keine Samples mit; gm_*-Sounds brauchen ein Extrapaket und bleiben draussen.
23      const SONIFIKATION_SAMPLE_BAENKE = [
24          ['https://strudel.b-cdn.net/piano.json', 'https://strudel.b-cdn.net/piano/'],
25          ['https://strudel.b-cdn.net/vcsl.json', 'https://strudel.b-cdn.net/VCSL/'],
26      ];
27
28      // VCSL hat keine Streicher/Klarinetten – Klavier/Orgel/Saxophon statt der
29      // ursprünglich gedachten Saint-Saëns-Besetzung. Was die Bank sehr wohl hat:
30      // Harfe, Xylophon, Glockenspiel, Pauke, Glasharmonika, Orgelpedal. Das
31      // Elementmodell weiter unten spielt darauf.
32      const SONIFIKATION_INSTRUMENTE = {
33          ort_loest_emotion_aus: { sound: 'piano', attack: 0.02, release: 0.6, octave: 3 },
34          emotion_faerbt_raum: { sound: 'pipeorgan_quiet', attack: 0.25, release: 1.2, octave: 4 },
35          koerper_als_sensor: { sound: 'sax', attack: 0.12, release: 0.8, octave: 4 },
36      };
37
38      // Gesamtdauer des Stücks – bewusst hier (nicht in Python) als gestalterischer
39      // Wert; erste Annahme, per Ohr anzupassen.
40      const SONIFIKATION_GESAMTDAUER_SEK = 45;
41
42      // Dauer je Station: Basiswert plus Anteile aus Gehstrecke und Annotationen.
43      // Die beiden Skalen sind Stellschrauben, keine gemessenen Grössen.
44      const SONIFIKATION_GEWICHT_BASIS = 3;
45      const SONIFIKATION_GEWICHT_STRECKEN_SKALA = 200; // Meter pro Gewichtspunkt
46      const SONIFIKATION_GEWICHT_ANNOTATION_SKALA = 0.6; // Gewichtspunkte pro Annotation
47
48
49      // --- Prototyp: ein Klang je Element (nur Kapitel 1) -----
50      //
51      // Dreht die Zuordnung des Stationsmodells um. Dort tragen die drei
52      // F-WERT-TYPEN die Instrumente; hier tragen sie die drei GEFÜHLSKATEGORIEN,
53      // und die F-Werte bekommen als Instrument 4 eine eigene Stimme. Damit klingt
54      // die Tonebene so, wie die Grafik gebaut ist: die Bänder sind die
55      // Kategorien, die Punkte am Rand sind die F-Werte.
56      //
57      // Jedes Element klingt einmal. Die Tonhöhe folgt dem Kreisradius, den es
58      // gerade erzeugt – und weil der Radius mit der Wurzel wächst, springt ein
59      // junger Kreis hörbar, während ein voller nur noch kriecht.
60
61      // Umschalter zum Vergleichen: 'stationen' spielt die bisherige Fassung –
62      // Kapitel 1 aus kapitel01-sonifikation.json, 02–18 aus den Spine-Einträgen.
63      const SONIFIKATION_MODUS = 'elemente';
64
65      // Spieldauer im Elementmodell. Nicht die Zahl der Orte zählt, sondern die
66      // der Klänge: Kapitel 2 spielt an einem einzigen Ort und bekäme über die

```

```

67 // Ortsformel 2,6 s für 118 Elemente, also 45 Klänge je Sekunde.
68
69 // ACHTUNG die Dauer gilt auch für die Grafik. aktuelleGrafikAnimationDauer()
70 // (spine-horizontal.js) fragt hier an, sonst liefen Bild und Ton auseinander
71 // – im Elementmodell hängt jeder Klang an seinem Element.
72
73 // Die Wurzel hält die Spanne zusammen: linear bekäme Kapitel 5 mit seinen 321
74 // Elementen 96 s, so sind es 66. Kapitel 1 bleibt die Marke mit 45 s.
75 const ELEMENT_DAUER_BEZUG = 150; // Elemente in Kapitel 1
76
77 // Woran die Zeitpunkte hängen:
78 // 'spine' – an dem Moment, in dem das Element im Bild erscheint. Ton und
79 // Grafik laufen exakt synchron. Preis: die Spine verteilt ihre
80 // Zeit auf Orte, nicht auf Elemente. Die 50 Elemente der Folies
81 // Bergère fallen deshalb in die letzten 2,6 s.
82 // 'element' – alle Elemente gleich weit auseinander (0,3 s).
83 // 'puls' – Abstände auf ein Pulsraster gerundet, also ein echtes Metrum.
84
85 // ACHTUNG 'element' und 'puls' laufen dem Bild davon, und zwar deutlich: die
86 // Folies Bergère brauchen metrisch mindestens 10 s statt 2,6 s. Gemessen bis
87 // 12 s Versatz am Schluss. Beide sind zum Hören da, nicht zum Zeigen.
88 const SONIFIKATION_ZEITBASIS = 'spine';
89
90 // Nur für 'puls': Rasterweite in Sekunden.
91 const SONIFIKATION_PULS_SEK = 0.25;
92
93 // Nachklang nach dem letzten Element; die Spine steht dann schon still.
94 const SONIFIKATION_NACHKLANG_SEK = 2;
95
96 // Tonvorrat. g-moll ist die Tonart der «Danse macabre» (1874) – von dort
97 // kommt auch das Xylophon der F-Werte.
98
99 // ACHTUNG Skalenstufen statt freier MIDI-Werte: die Stufen tragen die
100 // Melodie, dafür fallen die Vierteltonschritte der ersten Fassung weg. Das
101 // ist der bewusste Tausch «feine Schritte gegen Melodie».
102 const ELEMENT_TONART = 'minor';
103
104 // Woran sich die Stufenleiter misst:
105 // 'kapitel' – am grössten Kreis des ganzen Kapitels. Ein kleiner Ort bleibt
106 // unten, ein grosser steigt hoch; Kreisgrösse ist über das ganze
107 // Stück hinweg hörbar. Preis: die meisten Orte in Kapitel 1
108 // haben zwei bis sieben Elemente je Band und bekommen damit zwei
109 // bis drei Stufen – zu wenig für eine Linie.
110 // 'ort' – an dem, was der Ort selbst erreicht, und die Phrase wird so
111 // lang wie er Elemente hat (höchstens eine Oktave). Jeder Besuch
112 // bekommt eine eigene Linie. Preis: gleiche Tonhöhe sagt nicht
113 // mehr gleiche Kreisgrösse.
114 const ELEMENT_STUFENBEZUG = 'ort';
115
116 // Instrument 1–3, je eine Gefühlskategorie. Angeschlagene Instrumente statt
117 // Wellenformen: sie klingen von selbst ab und lassen Luft zwischen den
118 // Tönen – ohne die gibt es keine Melodie, nur Fläche. Die Oktave folgt der
119 // Farbreihe, dunkles Band tief, helles Band hoch.
120 const ELEMENT_INSTRUMENTE = {
121   gold_dunkel: { sound: 'harp', oktave: 3 }, // Raum und Umwelt
122   gold_mittel: { sound: 'vibraphone_soft', oktave: 4 }, // Stimmung und Emotion
123   gold_hell: { sound: 'glockenspiel', oktave: 5 }, // Gesellschaft und Soziales
124 };
125
126 // Instrument 4: das Xylophon der «Danse macabre». Feste Stufen aus dem
127 // g-moll-Dreiklang, damit die 63 Anschläge zu jeder Melodie passen.
128 // persoenliche_sehnsucht fehlt in FWERT_PUNKTGROESSE und kommt in Kapitel 1
129 // genau einmal vor – der seltenste Ton des Stücks.
130 const ELEMENT_FWERT_SOUND = 'xylophone_soft_pp';
131 const ELEMENT_FWERT_OKTAVE = 5;
132 const ELEMENT_FWERT_GRAD = {

```

```
133   ort_loest_emotion_aus: 0,
134   emotion_faerbt_raum: 2,
135   koerper_als_sensor: 4,
136   persoenliche_sehnsucht: 7,
137 };
138
139 // Bassstimme unter der Melodie: die Pedalregister der Orgelsymphonie (1886,
140 // dasselbe Jahr wie der «Carnaval des animaux»). Sie trägt das Wummern, das
141 // die Schraffur allein nicht mehr leisten kann, seit die Kategorien
142 // angeschlagen statt gehalten spielen – ein Ton je Ortsbesuch, nicht je
143 // Element. Nur Dreiklangstufen, damit nichts gegen die Melodie steht.
144
145 // ACHTUNG maxSek muss bleiben: ein Ortsbesuch dauert in Kapitel 2 die vollen
146 // 41 s, und ein Sample hält das nicht – der Bass wäre nach dem ersten Ton
147 // weg. Ausserdem füllt das Nachschlagen die Stellen, an denen gar kein
148 // Element wächst: Kapitel 9 hat bei 36,8 s eine Lücke von 5,3 s, weil dort
149 // ein Ort mit einem einzigen Element eine volle Abschnittsdauer belegt.
150 const ELEMENT_BASS = { sound: 'pipeorgan_quiet_pedal', oktave: 2, grade: [0, 2, 4], maxSek: 3 };
151
152 // Stufenumfang und Klangfarbe je Rolle. Die Melodiestimmen bekommen eine
153 // Oktave: jeder Ort fängt unten an und steigt – das ergibt je Ort eine
154 // Phrase, in Kapitel 1 also vierzehn.
155 const ELEMENT_ROLLEN = {
156   melodie: { grade: [0, 7], attack: 0.01, release: 1.4, gain: 0.55, room: 0.35 },
157   neg: { grade: [0, 5], attack: 0.01, release: 2.4, gain: 0.45, room: 0.45 },
158   pos: { grade: [0, 7], attack: 0.01, release: 1.6, gain: 0.40, room: 0.30 },
159   fwert: { grade: [0, 0], attack: 0.01, release: 0.7, gain: 0.30, room: 0.25 },
160   bass: { grade: [0, 0], attack: 0.6, release: 3.5, gain: 0.32, room: 0.5 },
161 };
162
163 // Die schwere Hälfte hat ein eigenes Fell: Pauke statt Kategorieinstrument.
164 // Die helle bekommt die Glasharmonika aus dem «Aquarium».
165 const ELEMENT_NEG = { sound: 'timpani', oktave: 2 };
166 const ELEMENT_POS = { sound: 'wineglass', oktave: 5 };
167
168
169 let sonifikationDaten = null;
170 let sonifikationBereit = false;
171 let sonifikationSpieltGerade = false;
172
173 // Zeitplan in Sekunden ab Start, nur modulintern für den Audio-Aufbau.
174 let sonifikationSpielplan = null;
175
176 async function ladeSonifikationDaten() {
177   if (sonifikationDaten) return sonifikationDaten;
178   let antwort = await fetch('kapitel01-sonifikation.json');
179   sonifikationDaten = await antwort.json();
180   return sonifikationDaten;
181 }
182
183 function baueSpielplan(stationen) {
184   let gewichte = stationen.map(s =>
185     SONIFIKATION_GEWICHT_BASIS
186     + (s.wegstreckeVorherM + s.wegstreckeEigenM) / SONIFIKATION_GEWICHT_STRECKEN_SKALA
187     + s.anzahlAnnotationen * SONIFIKATION_GEWICHT_ANNOTATION_SKALA
188   );
189   let summeGewichte = gewichte.reduce((a, b) => a + b, 0);
190
191   let ende = 0;
192   let revealIndexVorher = 0;
193   return stationen.map((s, i) => {
194     let dauer = (gewichte[i] / summeGewichte) * SONIFIKATION_GESAMTDAUER_SEK;
195     let start = ende;
196     ende += dauer;
197     let eintrag = {
198       station: s.station, ort: s.ort, start, ende, dauer,
```

```

199     revealIndexVorher,
200     revealIndexEigen: s.revealIndexMax,
201   };
202   revealIndexVorher = s.revealIndexMax;
203   return eintrag;
204 });
205 }
206
207 // Gain-Folge je Kategorie, auf maxAnzahl normiert. Dieselben @-Gewichte wie
208 // die Notenfolge, sonst laufen Gain- und Notenwechsel auseinander.
209 function baueGainFolge(stationen, spielplan, kategorie, maxAnzahl) {
210   return stationen
211     .map((s, i) => {
212       let n = s.fWertAnteile[kategorie] || 0;
213       let wert = n > 0 ? (n / maxAnzahl).toFixed(2) : '0';
214       return `${wert}@${spielplan[i].dauer.toFixed(3)}`;
215     })
216     .join(' ');
217 }
218
219 // ACHTUNG initStrudel() muss im Klick-Handler laufen (Autoplay-Policy) und
220 // gibt in @strudel/web@1.0.3 nichts zurück – setcps/cpm sind von aussen nicht
221 // erreichbar. Tempo liegt deshalb fest bei cps=0.5, die Gesamtdauer wird
222 // über .slow() gesteuert.
223 const SONIFIKATION_STANDARD_CPS = 0.5;
224
225 async function stelleSonifikationBereit() {
226   if (sonifikationBereit) return;
227   await initStrudel({
228     prebake: () => Promise.all(
229       SONIFIKATION_SAMPLE_BAENKE.map(([json, basis]) => samples(json, basis, { prebake: true }))
230     ),
231   });
232   // ACHTUNG das Promise von initStrudel() kann einen Tick VOR der Registrierung
233   // von n/s/note auflösen – ein n(...) direkt danach wirft "n is not defined".
234   // Kurzes Polling statt festem sleep(). Nur per Test beobachtet, nicht
235   // dokumentiert.
236   // note() zusätzlich, weil der Prototyp darauf steht statt auf n().
237   let versuche = 0;
238   while ((typeof n !== 'function' || typeof note !== 'function') && versuche < 50) {
239     await new Promise(r => setTimeout(r, 10));
240     versuche++;
241   }
242   sonifikationBereit = true;
243 }
244
245 let sonifikationTimeoutId = null;
246
247 // Gemeinsamer Wiedergabe-Kern: beide Aufrufer bauen nur notenFolge und
248 // gainFolgen, gespielt wird hier auf denselben drei Layern.
249 function spieleSchichten(notenFolge, gainFolgenProKategorie, slowFaktor, gesamtdauerSek) {
250   let layers = Object.entries(SONIFIKATION_INSTRUMENTE).map(([kategorie, instr]) =>
251     n(notenFolge)
252       .scale(`c${instr.octave}:minor`)
253       .s(instr.sound)
254       .gain(gainFolgenProKategorie[kategorie])
255       .attack(instr.attack)
256       .release(instr.release)
257       .room(0.3)
258       .slow(slowFaktor)
259   );
260
261   starteWiedergabe(stack(...layers), gesamtdauerSek);
262 }
263
264 // Start plus Selbstabschaltung. Beide Modelle teilen sich das, damit der

```

```

265 // Play-Zustand an einer einzigen Stelle gesetzt wird.
266 function starteWiedergabe(pattern, gesamtdauerSek) {
267     pattern.play();
268
269     sonifikationSpieltGerade = true;
270
271     sonifikationTimeoutId = setTimeout(() => {
272         sonifikationTimeoutId = null;
273         beendeSonifikationAudio();
274     }, gesamtdauerSek * 1000);
275 }
276
277 // Wählt den Ton zur offenen Ansicht: Kapitel 1, wenn keines gezoomt ist.
278 // Ohne await, der Aufrufer wartet ohnehin nicht.
279 function spieleSonifikationFuer(kapitelNr) {
280     if (SONIFIKATION_MODUS === 'elemente') return spieleElementAudio(kapitelNr);
281     if (kapitelNr) return spieleKapitelSonifikationAudio(kapitelNr);
282     return spieleKapitel1SonifikationAudio();
283 }
284
285 // -----
286 // Elementmodell (Prototyp, Kapitel 1)
287 // -----
288
289 // Fortschritt 0..1 je Annotation: wo im Ablauf sie im Bild erscheint.
290 // Umkehrung der Interpolation aus zeichneSpineHorizontal(), dort wächst
291 // globalAnnIndex stückweise linear zwischen den rv-Werten der Spine.
292
293 // ACHTUNG in Fortschritt statt in Sekunden, weil die Spieldauer erst aus der
294 // Zahl der Elemente folgt – und die steht erst fest, wenn diese Zuordnung
295 // gelaufen ist. Sekunden macht daraus erst spieleElementAudio().
296
297 // ACHTUNG das Hochziehen des letzten rv-Werts muss mit: die Spine setzt ihn
298 // auf die letzte Annotation ("sonst wird der letzte Kreis nie voll"). Ohne
299 // dieselbe Zeile laufen Ton und Bild auf den letzten Metern auseinander.
300 function elementFortschritte(daten, eintraege) {
301     let anzahl = daten.annotationen.length;
302     let n = eintraege.length;
303     let rv = eintraege.map(e => e.rv);
304     rv[n - 1] = anzahl - 1;
305
306     let werte = [];
307     let seg = 0;
308     for (let ai = 0; ai < anzahl; ai++) {
309         if (SONIFIKATION_ZEITBASIS === 'element') {
310             werte[ai] = ai / Math.max(1, anzahl - 1);
311             continue;
312         }
313         if (n === 1) {
314             // Einziger Eintrag (Kapitel 2): die Spine interpoliert dort ab -1,
315             // damit der Kreis nicht sofort voll steht. Dieselbe Umkehrung.
316             werte[ai] = (ai + 1) / Math.max(1, rv[0] + 1);
317             continue;
318         }
319         while (seg < n - 2 && ai > rv[seg + 1]) seg++;
320         let spanne = rv[seg + 1] - rv[seg];
321         let position = seg + (spanne > 0 ? (ai - rv[seg]) / spanne : 0);
322         werte[ai] = Math.min(1, Math.max(0, position / (n - 1)));
323     }
324     return werte;
325 }
326
327 // Ein Eintrag je Element, in Erzählreihenfolge, mit dem Kreisstand, den es
328 // gerade erzeugt. Keine neue Datei nötig: alles steht in den
329 // kapitelXX-stationen.json, die auch das Bild liest.
330 let elementCache = {};

```

```
331
332 function elementeFuerKapitel(kapitelNr) {
333   let schluessel = kapitelNr || '01';
334   if (elementCache[schluessel]) return elementCache[schluessel];
335   let gebaut = baueKapitelElemente(kapitelNr);
336   // Nur Fertiges merken: sind die Spine-Daten noch nicht aufgebaut, soll der
337   // nächste Aufruf es erneut versuchen statt für immer null zu liefern.
338   if (gebaut) elementCache[schluessel] = gebaut;
339   return gebaut;
340 }
341
342 function baueKapitelElemente(kapitelNr) {
343   let daten = kapitelNr ? datenFuerKapitel(kapitelNr) : stationenData;
344   let eintraege = spineEintraegeFuer(kapitelNr);
345   if (!daten || !daten.annotationen || !eintraege || !eintraege.length) return null;
346
347   // sammleAnnotationenNachOrtBasis() gibt Objekte zurück, gebraucht wird die
348   // Erzählposition – einmal vorwärts aufgebaut statt 150-mal indexOf.
349   let aiVon = new Map();
350   daten.annotationen.forEach((a, ai) => aiVon.set(a, ai));
351
352   // Welcher Kreis nimmt welches Element auf. Genau die Zuordnung, die
353   // zeichneSpineHorizontal() für seine Kreise benutzt, samt Wohnung-Split
354   // und Gedanken-Orten – deshalb dieselben zwei Funktionen und keine eigene
355   // Ortslogik. Rückkehr-Einträge brauchen keinen eigenen Durchgang: der
356   // Filter des Erstbesuchs fängt auch die späteren Annotationen ein.
357   let letzteAi = daten.annotationen.length - 1;
358   let kreisVonAi = new Map();
359   eintraege.forEach((e, i) => {
360     if (e.typ !== 'location') return;
361     sammleAnnotationenNachOrtBasis(wohnungFilterFuerOrt(e.ortBasis), letzteAi, daten)
362       .forEach(a => { if (aiVon.has(a)) kreisVonAi.set(aiVon.get(a), i); });
363   });
364
365   let fortschritte = elementFortschritte(daten, eintraege);
366
367   // Laufender Stand je Kreis und Kategorie: dieselben Zahlen, aus denen
368   // zaehleBandCounts() im Bild die Radian macht. valenz -1/1 wie valenzBucket.
369   let stand = new Map();
370   let elemente = [];
371   daten.annotationen.forEach((a, ai) => {
372     if (!a.category || !kreisVonAi.has(ai)) return;
373     let kreis = kreisVonAi.get(ai);
374     if (!stand.has(kreis)) stand.set(kreis, {});
375     let proKategorie = stand.get(kreis);
376     let z = proKategorie[a.category] || (proKategorie[a.category] = { gesamt: 0, neg: 0, pos: 0 });
377     z.gesamt++;
378     if (a.valenz === -1) z.neg++;
379     if (a.valenz === 1) z.pos++;
380     let gesamtAmKreis = 0;
381     Object.keys(proKategorie).forEach(k => { gesamtAmKreis += proKategorie[k].gesamt; });
382     elemente.push({
383       fortschritt: fortschritte[ai],
384       kategorie: a.category,
385       kreis: kreis,
386       // Alle Kategorien zusammen: die Grösse, die der Kreis im Bild zeigt.
387       kreisGesamt: gesamtAmKreis,
388       // Radius NACH diesem Element: der Klang meldet den Stand, den er macht.
389       schraffur: kreisRadius(z.gesamt),
390       neg: a.valenz === -1 ? kreisRadius(z.neg) : 0,
391       pos: a.valenz === 1 ? kreisRadius(z.pos) : 0,
392       fWertType: a.hasFwert ? a.fWertType : null,
393     });
394   });
395   return elemente.length ? elemente : null;
396 }
```

```

397
398 // Radius -> Skalenstufe. Unten der Radius des ersten Elements, oben der
399 // grösste Radius, den DIESE Stimme wirklich erreicht.
400
401 // ACHTUNG je Stimme normiert, nicht global: sonst bliebe «Gesellschaft und
402 // Soziales» mit seinen elf Elementen in den untersten drei Stufen kleben und
403 // hätte gar keine Linie. Preis: gleich grosse Kreise zweier Kategorien
404 // klingen nicht gleich hoch.
405 function elementGrad(radius, bezug, grade) {
406   let r0 = kreisRadius(1);
407   // Bei 'ort' reicht die Phrase nur so weit, wie der Ort Elemente hat: sonst
408   // spränge ein Ort mit zwei Elementen über die ganze Oktave.
409   let oben = ELEMENT_STUFENBEZUG === 'ort'
410     ? grade[0] + Math.min(grade[1] - grade[0], Math.max(0, bezug.anzahl - 1))
411     : grade[1];
412   let t = bezug.max > r0 ? (radius - r0) / (bezug.max - r0) : 0;
413   return grade[0] + Math.round(Math.min(1, Math.max(0, t)) * (oben - grade[0]));
414 }
415
416 // Sieben Stimmen auf einem gemeinsamen Raster aus einem Schritt je Element;
417 // wo eine Stimme schweigt, steht '~'. Der Bass läuft daneben in eigenen
418 // Schritten – einer je Ortsbesuch, damit er wirklich stehen bleibt.
419 function baueElementStimmen(elemente, gewichte) {
420   // Bezugsgrößen je Stimme, bei 'ort' zusätzlich je Kreis.
421   let bezug = {};
422   let schluessel = (name, kreis) => ELEMENT_STUFENBEZUG === 'ort' ? name + '#' + kreis : name;
423   let merke = (name, kreis, r) => {
424     let b = bezug[schluessel(name, kreis)] || (bezug[schluessel(name, kreis)] = { max: 0, anzahl: 0
425 });
426     b.max = Math.max(b.max, r);
427     b.anzahl++;
428   };
429   elemente.forEach(e => {
430     merke('melodie_' + e.kategorie, e.kreis, e.schraffur);
431     if (e.neg > 0) merke('neg', e.kreis, e.neg);
432     if (e.pos > 0) merke('pos', e.kreis, e.pos);
433   });
434
435   let stimmen = {};
436   let hole = (name, sound, oktave, rolle) => {
437     if (!stimmen[name]) stimmen[name] = { sound, oktave, rolle, grade: elemente.map(() => '~') };
438     return stimmen[name];
439   };
440
441   elemente.forEach((e, i) => {
442     let instr = ELEMENT_INSTRUMENTE[e.kategorie];
443     if (!instr) return;
444     let name = 'melodie_' + e.kategorie;
445     hole(name, instr.sound, instr.oktave, 'melodie').grade[i] =
446       elementGrad(e.schraffur, bezug[schluessel(name, e.kreis)], ELEMENT_ROLLEN.melodie.grade);
447     // Negativ und positiv je EINE Stimme über alle Kategorien: ein Element
448     // hat genau eine Valenz, zwei Kategorien können sich im selben Schritt
449     // also nicht ins Gehege kommen.
450     if (e.neg > 0) {
451       hole('neg', ELEMENT_NEG.sound, ELEMENT_NEG.oktave, 'neg').grade[i] =
452         elementGrad(e.neg, bezug[schluessel('neg', e.kreis)], ELEMENT_ROLLEN.neg.grade);
453     }
454     if (e.pos > 0) {
455       hole('pos', ELEMENT_POS.sound, ELEMENT_POS.oktave, 'pos').grade[i] =
456         elementGrad(e.pos, bezug[schluessel('pos', e.kreis)], ELEMENT_ROLLEN.pos.grade);
457     }
458     let fwertGrad = e.fWertType ? ELEMENT_FWERT_GRAD[e.fWertType] : undefined;
459     if (fwertGrad !== undefined) {
460       hole('fwert', ELEMENT_FWERT_SOUND, ELEMENT_FWERT_OKTAVE, 'fwert').grade[i] = fwertGrad;
461     }
462   });

```



```

462
463 // Ein Bassschritt je zusammenhängendem Besuch. Die Stufe kommt aus der
464 // Kreisgröße am Ende des Besuchs – das ist der Stand, den das Bild beim
465 // Weitergehen stehen lässt.
466 let maxGesamt = elemente.reduce((m, e) => Math.max(m, e.kreisGesamt), 1);
467 let bass = { grade: [], gewichte: [] };
468 elemente.forEach((e, i) => {
469   if (i > 0 && e.kreis === elemente[i - 1].kreis) {
470     bass.gewichte[bass.gewichte.length - 1] += gewichte[i];
471     bass.grade[bass.grade.length - 1] = ELEMENT_BASS.grade[
472       Math.min(ELEMENT_BASS.grade.length - 1,
473         Math.floor((e.kreisGesamt / maxGesamt) * ELEMENT_BASS.grade.length));
474     return;
475   }
476   bass.grade.push(ELEMENT_BASS.grade[0]);
477   bass.gewichte.push(gewichte[i]);
478 });
479
480 // Lange Besuche in gleich lange Anschläge zerlegen, Stufe bleibt: ein
481 // Orgelpunkt, der weiteratmet. Die Summe ändert sich dabei nicht, das
482 // Raster der übrigen Stimmen bleibt also unberührt.
483 let orgelpunkt = { grade: [], gewichte: [] };
484 bass.gewichte.forEach((w, i) => {
485   let teile = Math.max(1, Math.ceil(w / ELEMENT_BASS.maxSek));
486   for (let k = 0; k < teile; k++) {
487     orgelpunkt.grade.push(bass.grade[i]);
488     orgelpunkt.gewichte.push(w / teile);
489   }
490 });
491
492 return { stimmen, bass: orgelpunkt };
493 }
494
495 // n() + scale() statt note(): die Skalenstufen halten alle sieben Stimmen in
496 // derselben Tonart, und gebrochene MIDI-Werte braucht es damit nicht mehr.
497 function elementSchicht(grade, gewichte, sound, oktave, rolle, slowFaktor) {
498   return n(grade.map((wert, i) => wert + '@' + gewichte[i].toFixed(3)).join(' '))
499     .scale('g' + oktave + ':' + ELEMENT_TONART)
500     .s(sound)
501     .gain(rolle.gain)
502     .attack(rolle.attack)
503     .release(rolle.release)
504     .room(rolle.room)
505     .slow(slowFaktor);
506 }
507
508 // Spieldauer aus der Zahl der Elemente, siehe ELEMENT_DAUER_BEZUG.
509 function elementDauerSek(kapitelNr) {
510   let elemente = elementeFuerKapitel(kapitelNr);
511   if (!elemente) return null;
512   return SONIFIKATION_GESAMTDAUER_SEK * Math.sqrt(elemente.length / ELEMENT_DAUER_BEZUG);
513 }
514
515 // Für spine-horizontal.js: dieselbe Dauer in Millisekunden, oder null, wenn
516 // das Elementmodell nicht zuständig ist – dann gilt dort die Ortsformel.
517 function sonifikationElementDauerMs(kapitelNr) {
518   if (SONIFIKATION_MODUS !== 'elemente') return null;
519   stelleSpineDatenBereit(kapitelNr || undefined);
520   let sek = elementDauerSek(kapitelNr);
521   return sek ? sek * 1000 : null;
522 }
523
524 // Fortschritt mal Dauer ergibt Sekunden. Das Pulsraster greift erst hier,
525 // weil es in Sekunden misst und die Dauer vorher nicht feststeht.
526 function elementZeiten(elemente, gesamtdauerSek) {
527   let zeiten = elemente.map(e => e.fortschritt * gesamtdauerSek);

```



```

528   if (SONIFIKATION_ZEITBASIS !== 'puls') return zeiten;
529
530   // Jeden Abstand auf ganze Pulse runden, mindestens einen. Bewusst ohne
531   // Rückskalierung: die Stauchung würde den Versatz nur woandershin
532   // schieben, statt ihn hörbar zu lassen.
533   let gerastert = [zeiten[0]];
534   for (let i = 1; i < zeiten.length; i++) {
535     let schritte = Math.max(1, Math.round((zeiten[i] - zeiten[i - 1]) / SONIFIKATION_PULS_SEK));
536     gerastert[i] = gerastert[i - 1] + schritte * SONIFIKATION_PULS_SEK;
537   }
538   return gerastert;
539 }
540
541 // Ohne Kapitelnummer Kapitel 1, sonst das gezoomte – wie überall sonst.
542 async function spieleElementAudio(kapitelNr) {
543   await stelleSonifikationBereit();
544   stelleSpineDatenBereit(kapitelNr || undefined);
545   let elemente = elementeFuerKapitel(kapitelNr);
546   if (!elemente) return;
547
548   // Schrittgewichte in Sekunden: Abstand zum nächsten Element, der letzte
549   // Schritt trägt den Nachklang. Das kumulierte Gewicht eines Schritts ist
550   // damit genau sein Zeitpunkt – deshalb stimmt die Synchronität ohne
551   // weiteres Zutun. Die Untergrenze fängt gleichzeitige Elemente ab.
552   let zeiten = elementZeiten(elemente, elementDauerSek(kapitelNr));
553   let gewichte = zeiten.map((z, i) =>
554     i + 1 < zeiten.length ? Math.max(0.02, zeiten[i + 1] - z) : SONIFIKATION_NACHKLANG_SEK);
555   let gesamtdauerSek = gewichte.reduce((a, b) => a + b, 0);
556   let slowFaktor = gesamtdauerSek / (1 / SONIFIKATION_STANDARD_CPS);
557
558   let gebaut = baueElementStimmen(elemente, gewichte);
559   let layers = Object.values(gebaut.stimmen).map(st =>
560     elementSchicht(st.grade, gewichte, st.sound, st.oktave, ELEMENT_ROLLEN[st.rolle], slowFaktor));
561   layers.push(elementSchicht(gebaut.bass.grade, gebaut.bass.gewichte,
562     ELEMENT_BASS.sound, ELEMENT_BASS.oktave, ELEMENT_ROLLEN.bass, slowFaktor));
563
564   starteWiedergabe(stack(...layers), gesamtdauerSek);
565 }
566
567
568
569 // Reiner Audio-Start. Die Spine läuft unabhängig parallel und nutzt dieselbe
570 // SONIFIKATION_GESAMTDAUER_SEK, deshalb bleiben beide Uhren synchron.
571 async function spieleKapitel1SonifikationAudio() {
572   await stelleSonifikationBereit();
573   let daten = await ladeSonifikationDaten();
574   let stationen = daten.stationen;
575   let maxAnzahl = Math.max(...stationen.map(s => s.anzahlAnnotationen));
576
577   sonifikationSpielplan = baueSpielplan(stationen);
578
579   // Eine Tonstufe je Station, Länge über die @-Gewichte aus baueSpielplan.
580   // .slow() dehnt den einen Zyklus auf SONIFIKATION_GESAMTDAUER_SEK.
581   let notenFolge = sonifikationSpielplan.map((e, i) => `${i}@${e.dauer.toFixed(3)}').join(' ');
582   let slowFaktor = SONIFIKATION_GESAMTDAUER_SEK / (1 / SONIFIKATION_STANDARD_CPS);
583
584   let gainFolgenProKategorie = {};
585   Object.keys(SONIFIKATION_INSTRUMENTE).forEach(kategorie => {
586     gainFolgenProKategorie[kategorie] = baueGainFolge(stationen, sonifikationSpielplan, kategorie,
587       maxAnzahl);
588   });
589
590   spieleSchichten(notenFolge, gainFolgenProKategorie, slowFaktor, SONIFIKATION_GESAMTDAUER_SEK);
591 }
592
593 // Kapitel 02–18 lesen dieselben Spine-Einträge wie die Graph-Ansicht, Ton

```

```

593 // und Bild teilen so die Struktur ohne eigene Python-Datei.
594
595 // 'rueckkehr'-Schritte bleiben stumm ('~'): eine Rückkehr lässt den alten
596 // Kreis weiterwachsen, was sich sequenziell nicht nachbilden liesse.
597 async function spieleKapitelSonifikationAudio(nr) {
598     await stelleSonifikationBereit();
599     let daten = datenFuerKapitel(nr);
600     let eintraege = spineEintraegeFuer(nr);
601     if (!daten || !eintraege || !eintraege.length) return;
602
603     let annotationen = daten.annotationen;
604     let fWertAnteileJeSchritt = eintraege.map((e, j) => {
605         let bis = j + 1 < eintraege.length ? eintraege[j + 1].rv - 1 : annotationen.length - 1;
606         let anteile = { ort_loest_emotion_aus: 0, emotion_faerbt_raum: 0, koerper_als_sensor: 0 };
607         for (let ai = e.rv; ai <= bis; ai++) {
608             let a = annotationen[ai];
609             if (a && a.hasFwert && a.fWertType in anteile) anteile[a.fWertType]++;
610         }
611         return { anteile, anzahl: Math.max(0, bis - e.rv + 1) };
612     });
613     let maxAnzahl = Math.max(1, ...fWertAnteileJeSchritt.map(s => s.anzahl));
614
615     let melodieIndex = 0;
616     let notenFolge = eintraege.map(e => e.typ === 'rueckkehr' ? '~' : String(melodieIndex++)).join('');
617
618     let gainFolgenProKategorie = {};
619     Object.keys(SONIFIKATION_INSTRUMENTE).forEach(kategorie => {
620         gainFolgenProKategorie[kategorie] = eintraege.map((e, j) =>
621             e.typ === 'rueckkehr' ? '0' : (fWertAnteileJeSchritt[j].anteile[kategorie] /
maxAnzahl).toFixed(2)
622         ).join(' ');
623     });
624
625     // aktuelleGrafikAnimationDauer() (spine-horizontal.js) liest zoomedKapitel
626     // selbst – hier korrekt, weil der Aufruf nur aus toggleGrafikPlay kommt.
627     let gesamtdauerSek = aktuelleGrafikAnimationDauer() / 1000;
628     let slowFaktor = gesamtdauerSek / (1 / SONIFIKATION_STANDARD_CPS);
629
630     spieleSchichten(notenFolge, gainFolgenProKategorie, slowFaktor, gesamtdauerSek);
631 }
632
633 // Nur der Audio-Teil. Play-Zustand und Button gehören der Graph-Ansicht
634 // (toggleGrafikPlay, spine-horizontal.js) und bleiben unangetastet.
635 function beendeSonifikationAudio() {
636     if (typeof hush === 'function') hush();
637     sonifikationSpieltGerade = false;
638     if (sonifikationTimeoutId !== null) {
639         clearTimeout(sonifikationTimeoutId);
640         sonifikationTimeoutId = null;
641     }
642 }
643
644
645 // --- Export -----
646 // Fünf Namen. Leser: docs/architektur.md.
647 window.SONIFIKATION_GESAMTDAUER_SEK = SONIFIKATION_GESAMTDAUER_SEK;
648 window.spieleSonifikationFuer = spieleSonifikationFuer;
649 window.sonifikationElementDauerMs = sonifikationElementDauerMs;
650 window.beendeSonifikationAudio = beendeSonifikationAudio;
651
652 // Lesebindung statt Wertkopie: die Flagge fröre sonst auf false ein und der
653 // Ton liefе beim Ansichtswechsel weiter.
654 Object.defineProperty(window, 'sonifikationSpieltGerade', {
655     get: function () { return sonifikationSpieltGerade; },
656     configurable: true,

```

```
657 | });  
658 |  
659 | }()); // Ende der Modulkapselung, siehe Kommentar oben
```